

Zitadel for Developers — sfg-labs

How to authenticate against the sfg-labs platform. **Auth happens once, at the gateway.** You get a token from Zitadel; the APISIX gateway validates it and forwards your identity to backend services as headers. Backends contain **zero auth code** — they trust the gateway.

Auth server (Zitadel)	<code>https://sfg-labs.faithandgamble.in</code>
Admin console	<code>https://sfg-labs.faithandgamble.in/ui/console</code>
OIDC discovery	<code>https://sfg-labs.faithandgamble.in/.well-known/openid-configuration</code>
Token endpoint	<code>https://sfg-labs.faithandgamble.in/oauth/v2/token</code>
API gateway base	<code>https://sfg-labs.faithandgamble.in</code> (per-service hosts/paths)

How a request flows



Syntax error in text
mermaid version 11.15.0

Key rule: if your token is missing or invalid, you get `401` and the backend is never reached.

1. Get credentials (in the console)

 ZITADEL

DocumentationDefault settings  

HomeOrganizationProjectsUsersRole AssignmentsActionsSettings

Get started with Zitadel

Integrate Zitadel into your application

Integrate Zitadel into your application or use one of our samples to get started in minutes.

Create ApplicationLearn More



YOUR NEXT STEPS

 Register your application
Register your web, native

 Log in to your application
Integrate your application

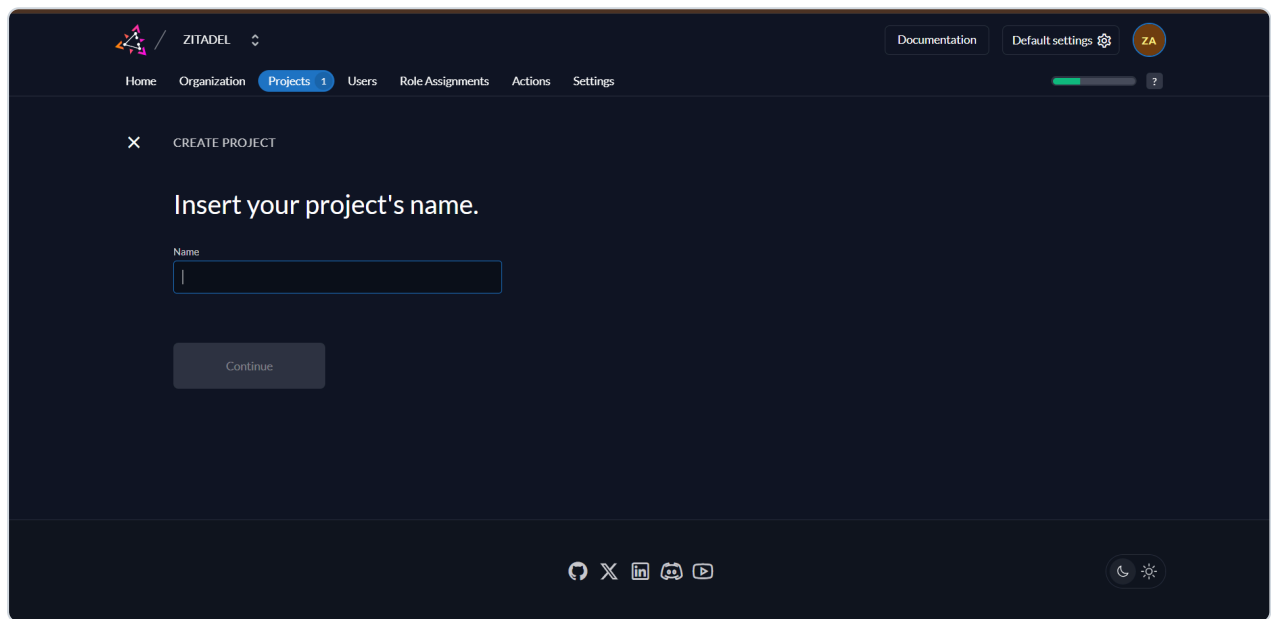
 Create a project
Add a project and define its roles and role assignments

Two common app types — pick based on what you're building:

You're building...	Create this in Zitadel	Auth method
A backend service / cron / machine-to-machine	Service User + key	JWT Profile (<code>jwt-bearer</code>) or client credentials
A web / SPA / mobile app with user login	Application (Web / SPA / Native) in a Project	OIDC Authorization Code + PKCE

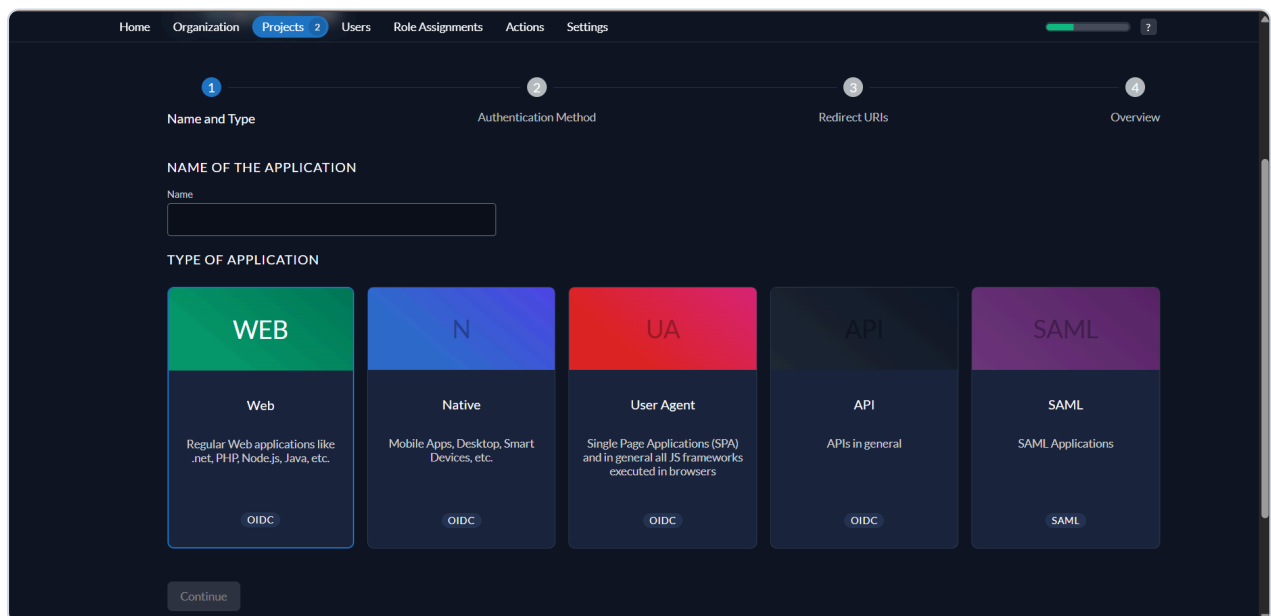
Create an API Application + Service User (machine-to-machine)

1. Console → **Projects** → **Create Project** (e.g. `my-service`).



The screenshot shows the 'CREATE PROJECT' screen in the Zitadel console. The top navigation bar includes 'Home', 'Organization', 'Projects 1', 'Users', 'Role Assignments', 'Actions', and 'Settings'. The main heading is 'CREATE PROJECT' with a close button. Below it, the instruction 'Insert your project's name.' is followed by a text input field labeled 'Name'. A 'Continue' button is positioned below the input field. The bottom of the screen features social media icons and a theme toggle.

2. Inside the project → **Applications** → **New**. Choose the application **type** — pick **API** for machine-to-machine:



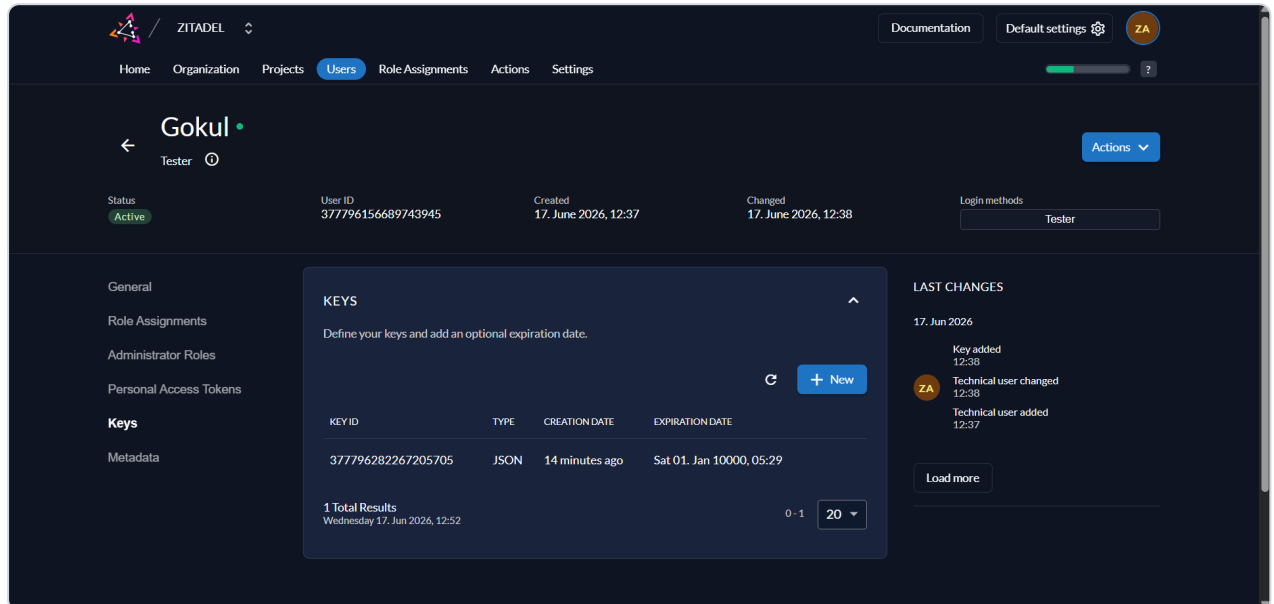
The screenshot shows the 'New Application' screen in the Zitadel console. The top navigation bar is the same as the previous screen. The main heading is 'NEW APPLICATION' with a close button. Below it, the instruction 'NAME OF THE APPLICATION' is followed by a text input field labeled 'Name'. Below that, the instruction 'TYPE OF APPLICATION' is followed by five colored cards: 'WEB' (green), 'N' (blue), 'UA' (red), 'API' (dark blue), and 'SAML' (purple). Each card has a sub-label and a description. The 'API' card is selected. A 'Continue' button is at the bottom left.

TYPE OF APPLICATION	Sub-label	Description	Auth Method
WEB	Web	Regular Web applications like .net, PHP, Node.js, Java, etc.	OIDC
N	Native	Mobile Apps, Desktop, Smart Devices, etc.	OIDC
UA	User Agent	Single Page Applications (SPA) and in general all JS frameworks executed in browsers	OIDC
API	API	APIs in general	OIDC
SAML	SAML	SAML Applications	SAML

- Set **Auth Method** = **Private Key JWT**.

- ⚠ In the application's token settings, set **Auth Token Type = JWT** (not "Bearer"). The gateway validates JWTs via JWKS and **cannot read opaque tokens** — this is the #1 cause of `401` s with an otherwise-valid token.
3. Console → **Users** → **Service Users** → **New**. Open the user → **Keys** → **New** → download the **JSON key** (you only see it once). It looks like:

```
{ "type": "serviceaccount", "keyId": " ... ", "key": "-----BEGIN RSA PRIVATE KEY----- ... ", "userId": " ... " }
```



2. Get a token

Machine-to-machine (service-account key → JWT Profile)

You sign a short JWT "assertion" with your private key and exchange it for an access token. Minimal Node script (no dependencies):

```
// node get-token.js key.json
const fs = require('fs'), crypto = require('crypto'), https = require('https');
const sa = JSON.parse(fs.readFileSync(process.argv[2], 'utf8'));
const DOMAIN = 'https://sfg-labs.faithandgamble.in';
const PROJECT_ID = ''; // set to include project roles in the token (see §3)
const now = Math.floor(Date.now() / 1000);
const b64 = (b) => Buffer.from(b).toString('base64url');

const input = b64(JSON.stringify({ alg: 'RS256', kid: sa.keyId })) + '.' +
  b64(JSON.stringify({ iss: sa.userId, sub: sa.userId, aud: DOMAIN, iat: now, exp: now + 3600 }));
const assertion = input + '.' + b64(crypto.createSign('RSA-SHA256').update(input).sign(sa.key));

let scope = 'openid profile';
if (PROJECT_ID) scope += ` urn:zitadel:iam:org:project:id:${PROJECT_ID}:aud`;

const body = new URLSearchParams({ grant_type: 'urn:ietf:params:oauth:grant-type:jwt-bearer', assertion, scope });
const u = new URL(DOMAIN + '/oauth/v2/token');
const req = https.request({ hostname: u.hostname, path: u.pathname, method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' } },
  (res) => { let d = ''; res.on('data', c => d += c); res.on('end', () => console.log(d)); });
req.end(body);
```

Run it:

```
node get-token.js 377796282267205705.json
```

Response contains `access_token` — a JWT you put in `Authorization: Bearer <token>`.

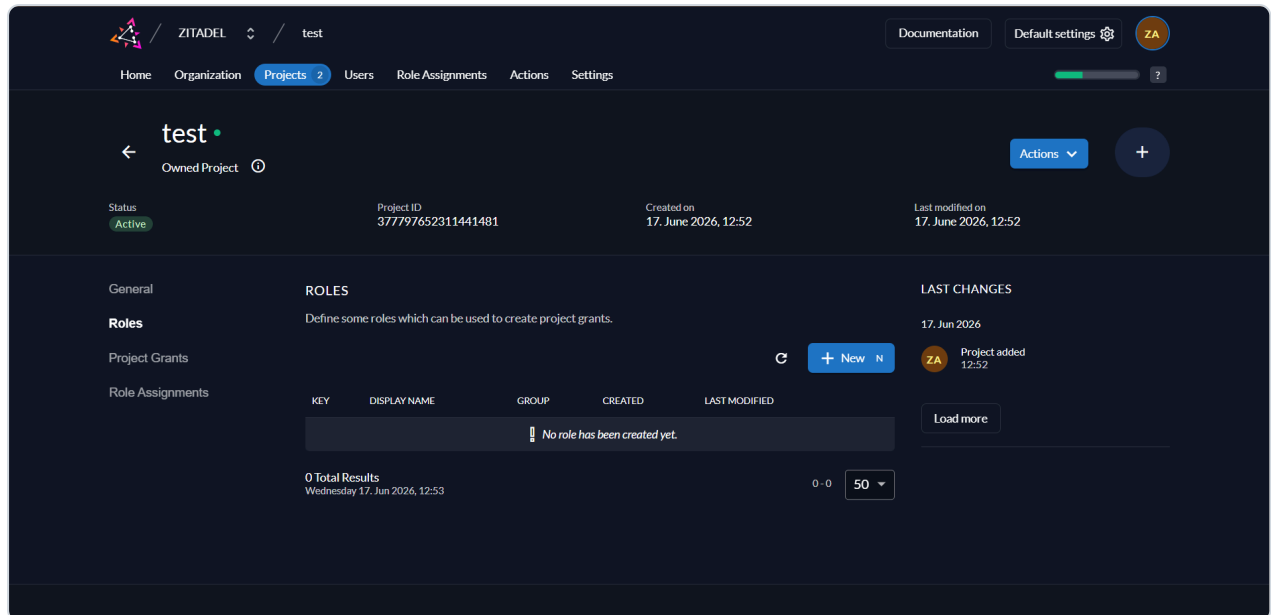
User-facing apps (Authorization Code + PKCE)

Use a standard OIDC library (e.g. `oidc-client-ts`, `next-auth`, `AppAuth`). Point it at the discovery URL above; the library handles redirect, PKCE, and token refresh. Register the app's redirect URI in the console under the Application's **Redirect URIs**.

3. Roles (the authorization model)

Roles are defined **per Project** and granted to users. They ride inside the token.

1. Project → **Roles** → **+ New** → add roles (e.g. `admin`, `auditor`, `cashier`).



- Grant roles to a user: project → **Role Assignments** (also reachable from the top-nav **Role Assignments**) → pick the user + roles. A user can hold **multiple roles**.
- Request the project audience scope so the roles appear in the token:

```
scope = openid profile urn:zitadel:iam:org:project:id:<PROJECT_ID>:aud
```

The roles then show up in the token under:

```
"urn:zitadel:iam:org:project:roles": {  
  "admin": { "<orgId>": "<orgDomain>" },  
  "auditor": { "<orgId>": "<orgDomain>" }  
}
```

Decode any token at <https://jwt.io> to inspect its claims.

4. What backends receive (no auth code needed)

After the gateway validates the token it injects headers. Your service reads these and trusts them (the request already passed auth at the edge):

Header	Contents
X-Userinfo	Base64-encoded JSON of the user's OIDC userinfo (sub, email, roles, ...)
X-Request-Id	Correlation id, echoed in the response
X-Gateway	sfg-labs (proves the request came through the gateway)

In the sfg-labs backend services this is decoded for you by the `readUser` middleware from `@suwalka/common` into `req.user` (an `AuthUser`). **Do not** implement token validation in your service — if `X-Userinfo` is absent, the request bypassed the gateway and should be rejected `401`.

5. Calling a protected endpoint

```
# No token → 401 (gateway blocks; backend never hit)
curl -i https://sfg-labs.faithandgamble.in/api/<service>/<path>

# With token → 200, identity forwarded to the backend
curl -s -H "Authorization: Bearer <ACCESS_TOKEN>" \
  https://sfg-labs.faithandgamble.in/api/<service>/<path>
```

Troubleshooting

Symptom	Cause / fix
401 with a token that decodes fine	Token is opaque , not JWT → set the app's Auth Token Type = JWT and mint a new token
401 "issuer" / signature error	Token from a different issuer; must be <code>https://sfg-labs.faithandgamble.in</code>
Roles missing from token	Grant the roles to the user and request <code>urn:zitadel:iam:org:project:id:<PROJECT_ID>:aud</code> scope
404 Route Not Found	The path/host has no gateway route yet — ask the platform team to add one
429 Too Many Requests	Rate limited; back off

Screenshots

Console screenshots live in `docs/images/` . One optional shot is still missing — **granting a role to a user**: save it as `docs/images/grant-roles.png` and embed it under §3 step 2 with `![[Grant roles](images/grant-roles.png)` .